



SPRING 2026

SCALABLE AI SERVICES WITH AGENTIC AI, MLOPS & LLMOPS

CS5305/CS621 - PROGRAMMING ASSIGNMENT 5

CS5305/CS621

SPRING 2026
LUMS SBASSE



Scalable, Enterprise-Grade Agentic AI Game Development

Assignment Overview

In this advanced lab, you will evolve the Dino Runner development pipeline from Programming Assignment 4 into a production-ready, enterprise-grade system. While PA4 focused on a linear state machine, PA5 shifts the architecture toward ReAct (Reasoning + Acting) Agents that utilize specialized tools and are protected by a multi-layered Guardrail Middleware architecture. You will implement advanced memory management, safety filters, and LLMOps monitoring to ensure the system is scalable, secure, and cost-effective.

Prerequisites

Task 0: Workspace Initialization

Requirement: You MUST utilize Mosaic AI resources to complete this assignment. Ensure your environment is properly initialized before proceeding.

Task 1: ReAct Agent Architecture & Tool Integration [30 marks]

Objective: Transition the system from static nodes to autonomous ReAct agents capable of iterative reasoning and tool execution.

Task 1.1: ReAct Agent Definition

Instead of fixed transitions, implement the `Engineer` and `QA Engineer` as ReAct agents. These agents must utilize a `thought-action-observation` loop to determine autonomously whether they need to call a tool or yield a final response to the graph.

Task 1.2: Specialized Toolset

Develop a `CodeInterpreterTool`. The `Engineer` agent must call this tool to test and execute its own generated code before passing the payload to the `QA Engineer`.

Task 1.3: Tool Fallbacks

Implement a reliable Tool Fallback mechanism. If the primary code execution tool fails (e.g., due to a library conflict or syntax error), the agent must interpret the error and dynamically fix its code before forwarding the state to QA.

Task 2: Guardrails & Security Middleware [30 Marks]

Objective: Protect the system from adversarial inputs and ensure output compliance using deterministic rule-checks and Human-in-the-Loop (HITL) interventions.

Task 2.1: Personally Identifiable Information (PII) Filter

Implement a `PIIMiddleware` at the `Director` node. This guardrail must systematically redact any emails, passwords, or developer API keys that are accidentally included in the initial user prompt.

Task 2.2: Human-in-the-Loop (HITL) Workflow Guardrails

Implement a mandatory HITL node following the `Scorer`.

- If the generated code is acceptable, the Director (Human) can terminate the iteration.
- If unacceptable, the Director can inject custom feedback, routing the graph back to the `Architect` to revise the planning requirements for the next iteration.
- **Constraint:** Every individual agent must have a HITL middleware pause before transitioning to the next phase in the execution chain.

Task 3: Advanced Memory & Context Management [20 Marks]

Objective: Optimize the system's token consumption and state retention across prolonged execution cycles.

Task 3.1: Conversation Summarization Middleware

As iterations increase, the message history scales linearly, increasing both compute costs and latency. Implement middleware that automatically summarizes the `architect_messages` and `qa_feedback` once the token count exceeds a predefined threshold. The summary should retain only the essential "state of the game."

Constraint: You must execute a minimum of **three iterations** to demonstrate this summarization, even if the system achieves a desirable score earlier.

Task 3.2: Persistent Checkpointing

Integrate a `MemorySaver` to persist the graph's state. If the system crashes or times out during the Human-in-the-Loop approval phase, the state must be fully recoverable from the last successful node without restarting the pipeline.

Task 4: MLflow Experiment Tracking [20 Marks]

Objective: Utilize MLflow within Databricks to monitor agent performance, operational costs, and model groundedness.

Every iteration of the Dino Runner code development must be explicitly logged as an MLflow run. You are required to track the following metrics:

- **Latency:** The total time taken for each agent's reasoning cycle.
- **Groundedness:** A model-based evaluation score (LLM-as-a-judge) that quantitatively measures how accurately the Engineer's code reflects the Architect's initial system design.

Task 5: Final Deliverables

Zip everything into a folder (<roll_number>_pa5.zip). The folder must contain:

1. A Databricks notebook implementing the full LangGraph with ReAct agents and Middleware. Format: <roll_number>_pa5.ipynb
2. A JSON report showing at least one instance where a guardrail successfully blocked or redacted an input.
3. A video link of the final Python game code, now featuring advanced features:
 1. Basic functionality of PA4
 2. Day/Night changes.
 3. Clustered Obstacles.
 4. Speed increase of dino as score increases.
 5. Persistent High-Score Utilization across multiple application launches.
4. A high-resolution screenshot of your MLflow UI demonstrating the tracked metrics across multiple runs and iterations.